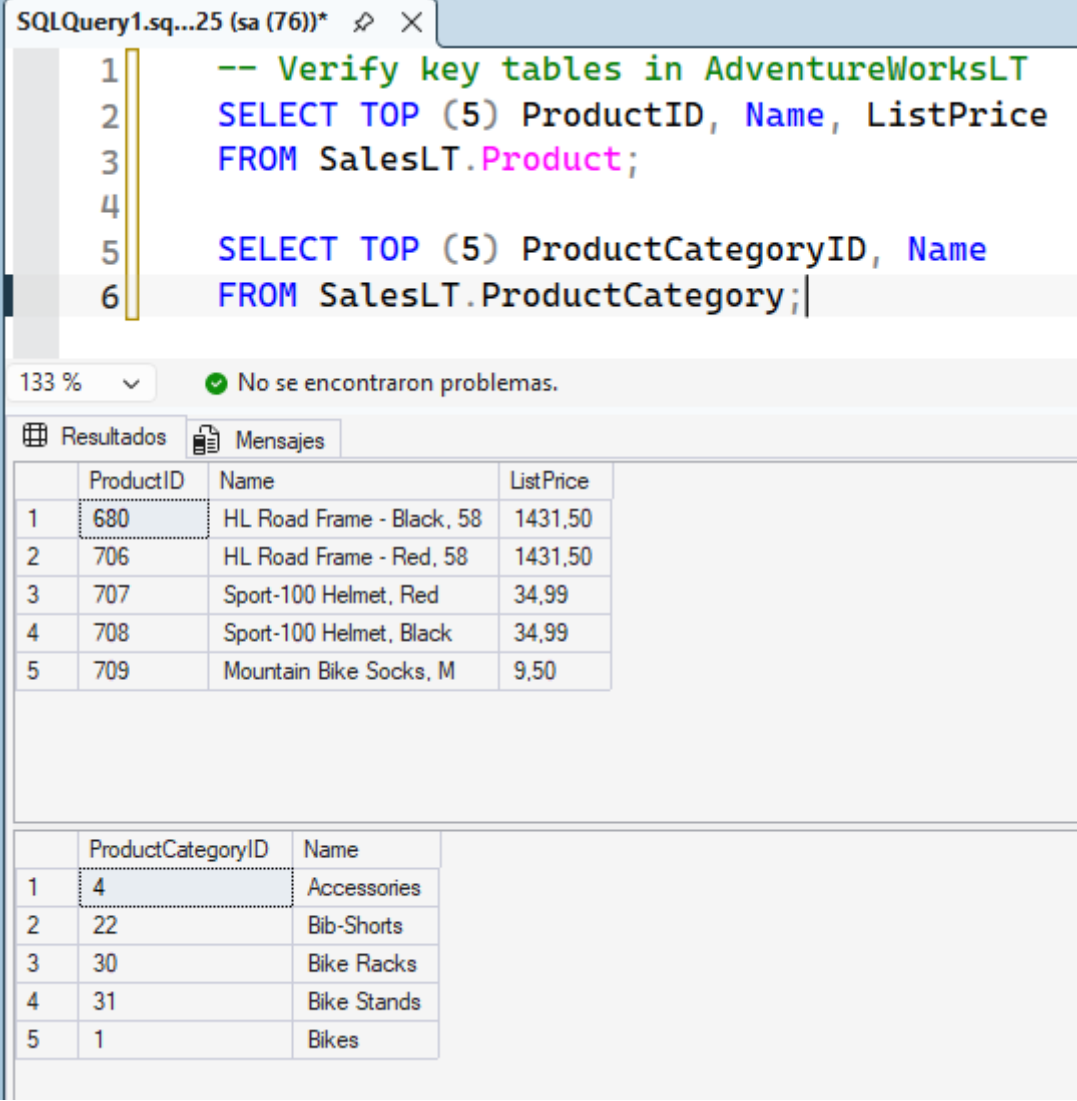


Write advanced T-SQL queries

Connect to AdventureWorksLT

Conexión a la base de datos AdventureWorksLT y verificación de las tablas clave: se consultan los 5 primeros registros de Product y ProductCategory para comprobar el acceso a los datos de productos y categorías.



The screenshot shows a SQL query window with the following text:

```
1  -- Verify key tables in AdventureWorksLT
2  SELECT TOP (5) ProductID, Name, ListPrice
3  FROM SalesLT.Product;
4
5  SELECT TOP (5) ProductCategoryID, Name
6  FROM SalesLT.ProductCategory;
```

Below the query, a status bar indicates "133 %" and "No se encontraron problemas." (No problems were found).

The results are displayed in two tables. The first table, titled "Resultados", shows the top 5 products from the Product table:

	ProductID	Name	ListPrice
1	680	HL Road Frame - Black, 58	1431,50
2	706	HL Road Frame - Red, 58	1431,50
3	707	Sport-100 Helmet, Red	34,99
4	708	Sport-100 Helmet, Black	34,99
5	709	Mountain Bike Socks, M	9,50

The second table, titled "Mensajes", shows the top 5 product categories from the ProductCategory table:

	ProductCategoryID	Name
1	4	Accessories
2	22	Bib-Shorts
3	30	Bike Racks
4	31	Bike Stands
5	1	Bikes

Build JSON output from product data

Generación de un objeto JSON simple a partir de la tabla Product utilizando FOR JSON PATH. La consulta selecciona productos con color asignado, los ordena de mayor a menor precio y formatea cada fila como un objeto dentro de un array JSON.

SQLQuery1.sql...25 (sa (76))*

```

1  SELECT
2      ProductID,
3      Name,
4      Color,
5      ListPrice
6  FROM SalesLT.Product
7  WHERE Color IS NOT NULL
8  ORDER BY ListPrice DESC
9  FOR JSON PATH;

```

133 % No se encontraron problemas.

Resultados Mensajes

	JSON_F52E2B61-18A1-11d1-B105-00805F49916B
1	[{"ProductID":749,"Name":"Road-150 Red, 62","Col...

Creación de una estructura JSON anidada mediante el uso de JSON_OBJECT. Se realiza un INNER JOIN con ProductCategory para agrupar el identificador y el nombre de la categoría dentro de su propia propiedad anidada (Category) en la salida final.

JSON_F52E2B61-18A1-11d1-B105-00805F49916B SQLQuery1.sql...25 (sa (76))*

```

1  SELECT
2      p.ProductID,
3      p.Name AS ProductName,
4      p.ListPrice,
5      JSON_OBJECT(
6          'CategoryID': pc.ProductCategoryID,
7          'CategoryName': pc.Name
8      ) AS Category
9  FROM SalesLT.Product AS p
10 INNER JOIN SalesLT.ProductCategory AS pc
11     ON p.ProductCategoryID = pc.ProductCategoryID
12 ORDER BY p.ListPrice DESC
13 FOR JSON PATH;

```

133 % No se encontraron problemas.

Resultados Mensajes

	JSON_F52E2B61-18A1-11d1-B105-00805F49916B
1	[{"ProductID":749,"ProductName":"Road-150 Red, 62","ListPrice":3578.2700,"Category":{"CategoryID":6,"CategoryNa...

Combine JSON with a CTE and window function

Uso de una expresión de tabla común (CTE) y la función de ventana ROW_NUMBER() para calcular el ranking de precios de los productos. PARTITION BY reinicia la numeración en cada categoría, asignando el valor 1 al producto más caro, lo que permite filtrar posteriormente los 3 productos principales de cada grupo.

```
SQLQuery1.sql...25 (sa (76))*
1  WITH RankedProducts AS (
2      SELECT
3          p.ProductID,
4          p.Name AS ProductName,
5          pc.Name AS CategoryName,
6          p.ListPrice,
7          ROW_NUMBER() OVER (
8              PARTITION BY pc.ProductCategoryID
9              ORDER BY p.ListPrice DESC
10         ) AS PriceRank
11  FROM SalesLT.Product AS p
12  INNER JOIN SalesLT.ProductCategory AS pc
13      ON p.ProductCategoryID = pc.ProductCategoryID
14  WHERE p.ListPrice > 0
15  )
16  SELECT
17      ProductID,
18      ProductName,
19      CategoryName,
20      ListPrice,
21      PriceRank
22  FROM RankedProducts
23  WHERE PriceRank <= 3
24  ORDER BY CategoryName, PriceRank;
```

133 % No se encontraron problemas.

Resultados Mensajes

	ProductID	ProductName	CategoryName	ListPrice	PriceRank
1	855	Men's Bib-Shorts, S	Bib-Shorts	89,99	1
2	856	Men's Bib-Shorts, M	Bib-Shorts	89,99	2
3	857	Men's Bib-Shorts, L	Bib-Shorts	89,99	3
4	876	Hitch Rack - 4-Bike	Bike Racks	120,00	1
5	879	All-Purpose Bike Stand	Bike Stands	159,00	1
6	871	Mountain Bottle Cage	Bottles and Cages	9,99	1
7	872	Road Bottle Cage	Bottles and Cages	8,99	2
8	870	Water Bottle - 30 oz.	Bottles and Cages	4,99	3
9	996	HL Bottom Bracket	Bottom Brackets	121,49	1

Formateo de los resultados del ranking directamente como JSON API agregando la cláusula FOR JSON PATH, ROOT('TopProducts'). Esto envuelve todo el array en un único objeto raíz, facilitando su consumo en aplicaciones externas.

```
SQLQuery1.sql...25 (sa (76))*
1  WITH RankedProducts AS (
2      SELECT
3          p.ProductID,
4          p.Name AS ProductName,
5          pc.Name AS CategoryName,
6          p.ListPrice,
7          ROW_NUMBER() OVER (
8              PARTITION BY pc.ProductCategoryID
9              ORDER BY p.ListPrice DESC
10         ) AS PriceRank
11     FROM SalesLT.Product AS p
12     INNER JOIN SalesLT.ProductCategory AS pc
13         ON p.ProductCategoryID = pc.ProductCategoryID
14     WHERE p.ListPrice > 0
15 )
16 SELECT
17     ProductID,
18     ProductName,
19     CategoryName,
20     ListPrice,
21     PriceRank
22 FROM RankedProducts
23 WHERE PriceRank <= 3
24 ORDER BY CategoryName, PriceRank
25 FOR JSON PATH, ROOT('TopProducts');
```

133 % No se encontraron problemas.

Resultados Mensajes

JSON_F52E2B61-18A1-11d1-B105-00805F49916B	
1	{ "TopProducts": [{ "ProductID": 855, "ProductName": "Men's Bib-Shorts, S", "CategoryName": "Bib-Shorts", "ListPri...

Parse JSON data with OPENJSON

Conversión de un array JSON de actualizaciones de precio en formato de filas y columnas utilizando la función OPENJSON. La cláusula WITH define el esquema, mapeando las rutas JSON (\$.ProductID, \$.NewPrice) a tipos de datos específicos de SQL Server.

SQLQuery1.sql...25 (sa (76))*

```
1 DECLARE @ProductUpdates NVARCHAR(MAX) = N'[
2     {"ProductID": 680, "NewPrice": 1250.00},
3     {"ProductID": 706, "NewPrice": 1450.00},
4     {"ProductID": 707, "NewPrice": 38.99}
5 ]';
6
7 SELECT
8     ProductID,
9     NewPrice
10 FROM OPENJSON(@ProductUpdates)
11 WITH (
12     ProductID INT '$.ProductID',
13     NewPrice DECIMAL(10,2) '$.NewPrice'
14 );
```

133 % No se encontraron problemas.

Resultados Mensajes

	ProductID	NewPrice
1	680	1250.00
2	706	1450.00
3	707	38.99

Integración de los datos JSON analizados con la tabla base Product. Dado que OPENJSON genera una estructura tabular, se puede ejecutar un INNER JOIN clásico para comparar el precio actual con la propuesta de actualización en tiempo real.

SQLQuery1.sql...25 (sa (76))*

```
1 DECLARE @ProductUpdates NVARCHAR(MAX) = N'[
2     {"ProductID": 680, "NewPrice": 1250.00},
3     {"ProductID": 706, "NewPrice": 1450.00},
4     {"ProductID": 707, "NewPrice": 38.99}
5 ]';
6
7 SELECT
8     p.ProductID,
9     p.Name,
10    p.ListPrice AS CurrentPrice,
11    updates.NewPrice,
12    updates.NewPrice - p.ListPrice AS PriceDifference
13 FROM SalesLT.Product AS p
14 INNER JOIN OPENJSON(@ProductUpdates)
15 WITH (
16     ProductID INT '$.ProductID',
17     NewPrice DECIMAL(10,2) '$.NewPrice'
18 ) AS updates
19 ON p.ProductID = updates.ProductID;
```

133 %

✓ No se encontraron problemas.

Resultados Mensajes

	ProductID	NewPrice
1	680	1250.00
2	706	1450.00
3	707	38.99